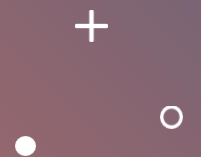


• Software Project Estimation Techniques

COCOMO, PERT, and Function Point Analysis



Software Project Estimation



Software estimation is the process of predicting the **effort, cost, time, and resources** required to develop a software system.



It helps project managers plan development activities and ensure projects are delivered within **budget and schedule**.



Accurate estimation is essential for successful **software project management**.

Why Estimation



Helps determine **project cost and budget**



Helps estimate project completion time
Helps allocate team members and resources



Supports project planning and scheduling

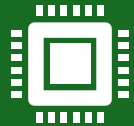


Helps reduce project risk and overruns

What is Estimation



Software estimation is the process of predicting the most realistic amount of effort, time, and cost required to build a software system.



Main Components : Effort, Time, Cost



When Estimation

- Estimation is performed at different stages of a software project.
- During project proposal stage
During requirement analysis
During project planning
Before starting development sprints
During project re-planning
- Estimation is not a one-time activity.
It is continuously updated as project requirements change.



Estimation Categories

Judgment-Based Techniques

Expert Judgment, Delphi Technique

Model-Based Techniques

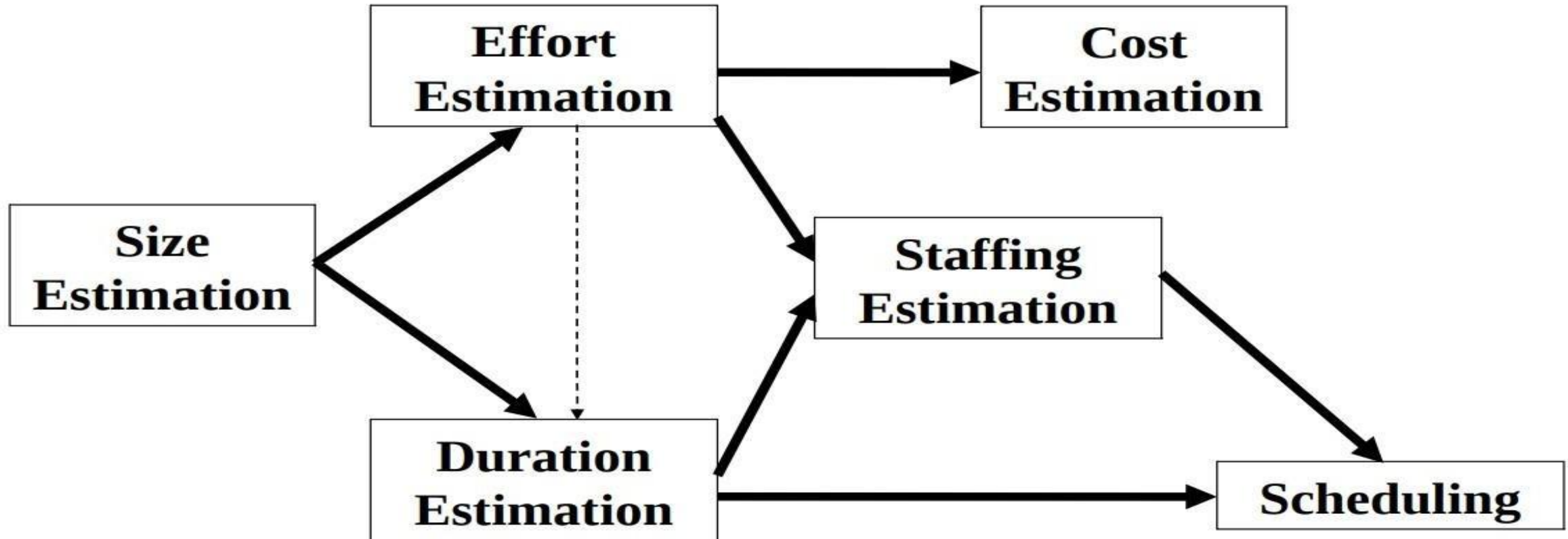
COCOMO, PERT

Metric-Based Techniques

Function Point Analysis

Story Points

Software Cost Estimation



Judgment-Based Techniques

- Expert Judgment

Expert Judgment is an estimation technique based on the **experience of senior developers or project managers.**

- Delphi Technique

Delphi technique is a structured method for obtaining **consensus among experts.**

Expert A → 8 weeks

Expert B → 10 weeks

Expert C → 12 weeks

Decision : 10 weeks(avg)

Analogous Estimation

- Analogous estimation is based on **comparison with previous similar projects**.
- The new project estimate is derived by adjusting estimates from past projects.
- Advantages:
 - Fast estimation
 - Useful in early stages
- Example:
- Previous mobile app → 5 months
- Estimated time = **6-7 months**

COCOMO

Introduction

COCOMO stands for **Constructive Cost Model**.

It was developed by **Barry Boehm** in **1981**.

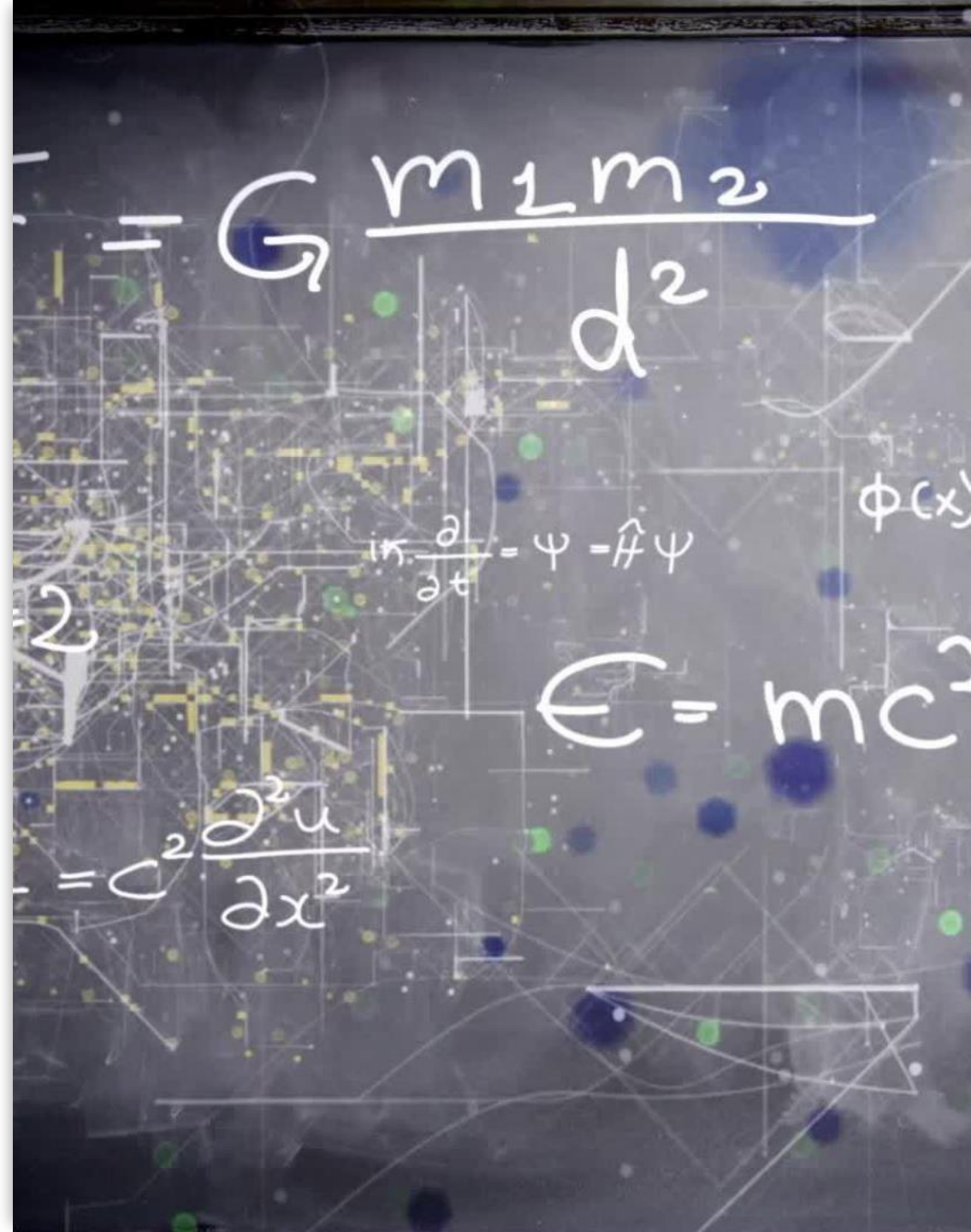
COCOMO estimates the **effort required to develop software** based on the **size of the program**.

Input:
Lines of Code (LOC)

Output:
Effort (Person-Months)

Basic COCOMO

- The Basic COCOMO model estimates effort using the formula:
- **Effort (PM) = $a \times (\text{KLOC})^b$**
- Where:
- KLOC = Thousands of Lines of Code
- Constants depend on project type.



Values of a and b as per types

Project Type	a	b
Organic	2.4	1.05
Semi-detached	3.0	1.12
Embedded	3.6	1.20

How to Estimate KLOC

- KLOC means **Thousands of Lines of Code**.
- Ways to estimate KLOC:
 - Use **expert opinions**
 - **Analyze requirement specifications**
 - **Use design documents**
 - **Estimate LOC for each module**
 - **Combine estimates from multiple experts**
- Average estimate from experts is often used for **better accuracy**.

COCOMO Numerical Example

- Consider an **office automation system**.
- Estimated modules:
 - Data Entry → 0.6 KDSI
 - Data Update → 0.6 KDSI
 - Query → 0.8 KDSI
 - Report Generation → 1.0 KDSI
- Total Size = **3 KDSI**
- The project is classified as **Organic type**.

COCOMO Solution

- Effort formula:
- $\text{Effort} = a \times (\text{KLOC})^b$
- For Organic projects:
- $a = 3.2$
 $b = 1.05$
- $\text{Effort} = 3.2 \times (3)^{1.05}$
 $\text{Effort} \approx 10.14 \text{ person-months}$
- After applying cost drivers:
- Final Effort $\approx 16.33 \text{ person-months}$

Intermediate COCOMO

- Intermediate COCOMO improves estimation accuracy by including **cost drivers**.
- Examples of cost drivers:
 - Product complexity
 - Developer experience
 - Hardware constraints
 - Development tools
- These factors adjust the basic effort estimation.

Example

Consider a database system needed for an office automation project. The requirements document shows 4 modules needed.

Sizes are estimated as follows:

data entry	0.6	KDSI
data update	0.6	KDSI
query	0.8	KDSI
report gen	1.0	KDSI

◦ How can SIZE be estimated in the first place?

From the requirements the ratings were assessed:

The manager rates project details as follows:

characteristic	level	EAF	
complexity	high	1.15	All others: nominal (1.0)
storage	high	1.06	
experience	low	1.13	
programmer capabilities	low	1.17	

Solution

system SIZE 3.0 KDSI

Intermediate
Cocomo

The project is judged to be ORGANIC (so $a=3.2$, $b=1.05$)

Initial effort: $3.2 * 3^{1.05} = 10.14$ PM (person-months)

Person Months for the project:

$$\begin{aligned} \text{PM} &= (1.15 * 1.06 * 1.13 * 1.17) * 3.2 * (3.0)^{1.05} \\ \text{PM} &= 1.61 * 3.2 * 3.17 \\ \text{PM} &= 16.33 \end{aligned}$$

Now it is an
Adjusted Effort

> 16 person months estimated

◦ How many people should we hire for this effort?

Solution ...cont'd

The model:

Organic	DURATION= 2.5 * EFFORT	0.38
Semi-Detached	DURATION= 2.5 * EFFORT	0.35
Embedded	DURATION= 2.5 * EFFORT	0.32

We had 16.33 person-months estimated for the database project.
The project was judged **ORGANIC**

$$\begin{aligned} \text{DURATION} &= 2.5 * (16.33) \\ \text{DURATION} &= 2.5 * 2.89 \\ \text{DURATION} &= 7.23 \end{aligned}$$

> 7 months to complete

How many people to hire?

$$\frac{16.33 \text{ person-months}}{7.23 \text{ months}} = 2.26 \text{ persons}$$

2 or 3 team members needed

Complete COCOMO

- In Complete COCOMO, the system is divided into **subsystems**.
- Each subsystem is estimated separately.
- Example system:
 - Database module
 - User interface module
 - Communication module
- Total project cost =
Sum of all subsystem costs.
- This approach **reduces estimation errors**.

PERT Explanation

- PERT stands for **Program Evaluation Review Technique**.
- It is used for **project scheduling and time estimation**.
- PERT considers three time estimates:
 - Optimistic (O)
 - Most Likely (M)
 - Pessimistic (P)
- Expected Time formula:
- $TE = (O + 4M + P) / 6$
- This provides a **weighted average estimate**.

Step 1: Break Project into Activities

Activity ID	Description	Predecessor(s)
A	Requirement Gathering	-
B	UI/UX Design	A
C	Backend Setup (APIs, DB, Server)	A
D	Android App Development	B, C
E	iOS App Development	B, C
F	Google Maps & GPS Integration	D, E
G	Testing & Bug Fixing	F
H	Final Deployment & Handover	G

Step 2: Estimate Time for Each Activity

For each activity, estimate the **Optimistic (O)**, **Most Likely (M)**, and **Pessimistic (P)** time (in weeks):

Activity	O	M	P
A	1	2	3
B	2	3	6
C	2	3	4
D	3	5	9
E	3	5	8
F	2	3	5
G	2	3	7
H	1	1	2



Step 3: Calculate Expected Time (TE) and Variance (σ^2)

Using PERT formula:

$$TE = (O + 4M + P) / 6$$

$$\text{Variance } (\sigma^2) = ((P - O)/6)^2$$

Activity	TE = (O+4M+P)/6	$\sigma^2 = ((P-O)/6)^2$
A	$(1+8+3)/6 = 2.0$	$(2/6)^2 = 0.11$
B	$(2+12+6)/6 = 3.33$	$(4/6)^2 = 0.44$
C	$(2+12+4)/6 = 3.0$	$(2/6)^2 = 0.11$
D	$(3+20+9)/6 = 5.33$	$(6/6)^2 = 1.00$
E	$(3+20+8)/6 = 5.17$	$(5/6)^2 = 0.69$
F	$(2+12+5)/6 = 3.0$	$(3/6)^2 = 0.25$
G	$(2+12+7)/6 = 3.17$	$(5/6)^2 = 0.69$
H	$(1+4+2)/6 = 1.17$	$(1/6)^2 = 0.03$

PERT Example

- Example:
- Task: Develop login module
- Optimistic time (O) = 2 weeks
Most likely time (M) = 4 weeks
Pessimistic time (P) = 6 weeks
- Expected time:
- $TE = (2 + 4 \times 4 + 6) / 6$
TE = 4 weeks

PERT vs COCOMO

Feature	PERT	COCOMO
Purpose	Time Estimation	Effort Estimation
Input	Time estimates (O, M, P)	LOC
Output	Duration (days/weeks)	Person-Months
Flexibility	Good for uncertainty	Good for structured projects

Critical Path

- The **Critical Path** is the longest path in a project network.
- It determines the **minimum project completion time**.
- Important properties:
 - Activities on critical path cannot be delayed
 - Any delay will delay the entire project
 - Helps project managers prioritize tasks



Step 4: Determine Critical Path

To identify the **Critical Path**, you need to draw the network diagram and calculate the **Longest Path**!

Let's summarize paths (only showing total durations here):

- **Path 1:** $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G \rightarrow H = 2 + 3.33 + 5.33 + 3.0 + 3.17 + 1.17 = 18 \text{ weeks}$
- **Path 2:** $A \rightarrow C \rightarrow E \rightarrow F \rightarrow G \rightarrow H = 2 + 3.0 + 5.17 + 3.0 + 3.17 + 1.17 = 17.51 \text{ weeks}$

 **Critical Path = Path 1 with 18 weeks**



Step 5: Project Variance and Standard Deviation

Variance is **sum of variances of activities on the critical path**:

Activities on Critical Path: A, B, D, F, G, H

→ Total Variance = $0.11 + 0.44 + 1.00 + 0.25 + 0.69 + 0.03 = \mathbf{2.52}$

→ Standard Deviation = $\sqrt{2.52} \approx \mathbf{1.59 \text{ weeks}}$



Step 6: Probability of Completion in 20 Weeks

We can use **Z-score** to estimate probability:

$$Z = (X - TE) / SD = (20 - 18) / 1.59 \approx \mathbf{1.26}$$

Using normal distribution table:

$P(Z \leq 1.26) \approx 0.8962 \rightarrow \mathbf{89.6\% \text{ chance}}$ of completing the project in **20 weeks**

Function Point Introduction

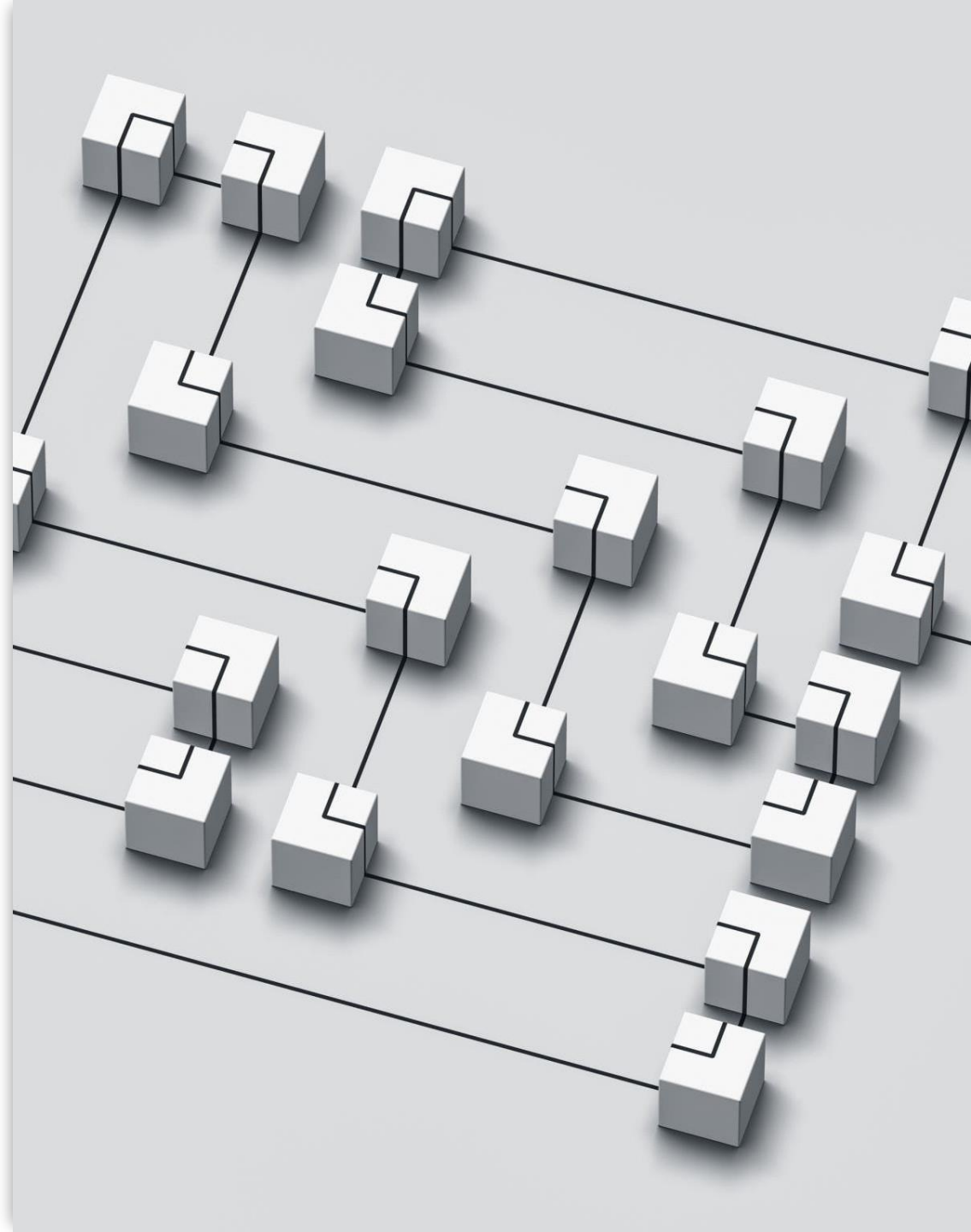
- Function Point Analysis (FPA) measures the **size of software based on functionality** rather than lines of code.
- It is useful when:
 - Programming language is unknown
 - LOC estimation is difficult
 - Early project estimation is required

Function Point Components

- Five major components are used:
- External Inputs (EI)
External Outputs (EO)
External Inquiries (EQ)
Internal Logical Files (ILF)
External Interface Files (EIF)
- These components represent the **functional requirements of the system.**

FP Diagram

- Function Point Analysis considers:
- Inputs entering the system
Outputs leaving the system
Internal logical files
External interfaces with other systems
- These elements together represent the **functional size of the software system.**



FP Formula

- Function Points are calculated using the formula:
- $FP = 4 \times \text{Inputs}$
- $5 \times \text{Outputs}$
- $4 \times \text{Inquiries}$
- $10 \times \text{Files}$
- $7 \times \text{Interfaces}$
- The calculated FP value represents the **functional size of the software**.

Steps in FP Computation

- Step 1. Classify each component of the product (*Inp, Out, Inq, Maf, Inf*) as simple, average, or complex (Figure 9.3)
 - Assign the appropriate number of function points
 - The sum gives UFP (unadjusted function points)

Component	Level of Complexity		
	Simple	Average	Complex
Input item	3	4	6
Output item	4	5	7
Inquiry	3	4	6
Master file	7	10	15
Interface	5	7	10

- Step 3. The number of function points (*FP*) is then given by

$$FP = UFP \times TCF$$

- Step 2. Compute the technical complexity factor (*TCF*)

- Assign a value from 0 ("not present") to 5 ("strong influence throughout") to each of 14 factors such as transaction rates, portability (Figure 9.4)

1. Data communication
2. Distributed data processing
3. Performance criteria
4. Heavily utilized hardware
5. High transaction rates
6. Online data entry
7. End-user efficiency
8. Online updating
9. Complex computations
10. Reusability
11. Ease of installation
12. Ease of operation
13. Portability
14. Maintainability

Copyright © The McGraw-Hill Companies, Inc.

- Sum the 14 numbers
 - This gives the total degree of influence (*DI*)

$$TCF = 0.65 + 0.01 \times DI$$

- The technical complexity factor (*TCF*) lies between 0.65 and 1.35

External Inputs (EI)

Files Type Referenced (FTR)	Data Elements		
	1-4	5-15	Greater than 15
Less than 2	Low (3)	Low (3)	Average (4)
2	Low (3)	Average (4)	High (6)
Greater than 2	Average (4)	High (6)	High (6)

External Outputs (EO)

File Types Referenced (FTR)	Data Elements		
	1-5	6-19	Greater than 19
less than 2	Low (4)	Low (4)	Average (5)
2 or 3	Low (4)	Average (5)	High (7)
Greater than 3	Average (5)	High (7)	High (7)

External Inquiry (EQ)

File Types Referenced (FTR)	Data Elements		
	1-5	6-19	Greater than 19
less than 2	Low (3)	Low (3)	Average (4)
2 or 3	Low (3)	Average (4)	High (6)
Greater than 3	Average (4)	High (6)	High (6)

Internal Logical Files (ILF)

Record Element Types (RET)	Data Elements		
	1 to 19	20 - 50	51 or More
1 RET	Low (7)	Low(7)	Average (10)

DET	<i>Data Element Type (DET)</i> : A DET is a unique user recognizable, non-recursive (non-repetitive) field.
EI	<i>External Inputs (EI)</i> - is an elementary process in which data crosses the boundary from outside to inside.
EIF	<i>External Interface Files (EIF)</i> - a user identifiable group of logically related data that is used for reference purposes only.
EO	<i>External Outputs (EO)</i> - an elementary process in which <i>derived data</i> passes across the boundary from inside to outside
EQ	<i>External Inquiry (EQ)</i> - an elementary process with both input and output components that result in data retrieval from one or more internal logical files and external interface files.
FTR	<i>File Type Referenced (FTR)</i> : A FTR is a file type referenced by a transaction. An FTR must also be an internal logical file or external interface file.
GSC	General System Characteristics
ILF	<i>Internal Logical Files (ILF)</i> - a user identifiable group of logically related data that resides entirely within the application boundary and is maintained through External Input
RET	<i>Record Element Type (RET)</i> : A RET is user recognizable sub group of data elements within an ILF or an EIF
UAF	Unadjusted Function Points
VAF	The <i>value adjustment factor (VAF)</i> is based on 14 general system characteristics
AFP	Application Function Point Count after the enhancement
AFPB	The initial application function point count or baseline

FP Calculation Steps

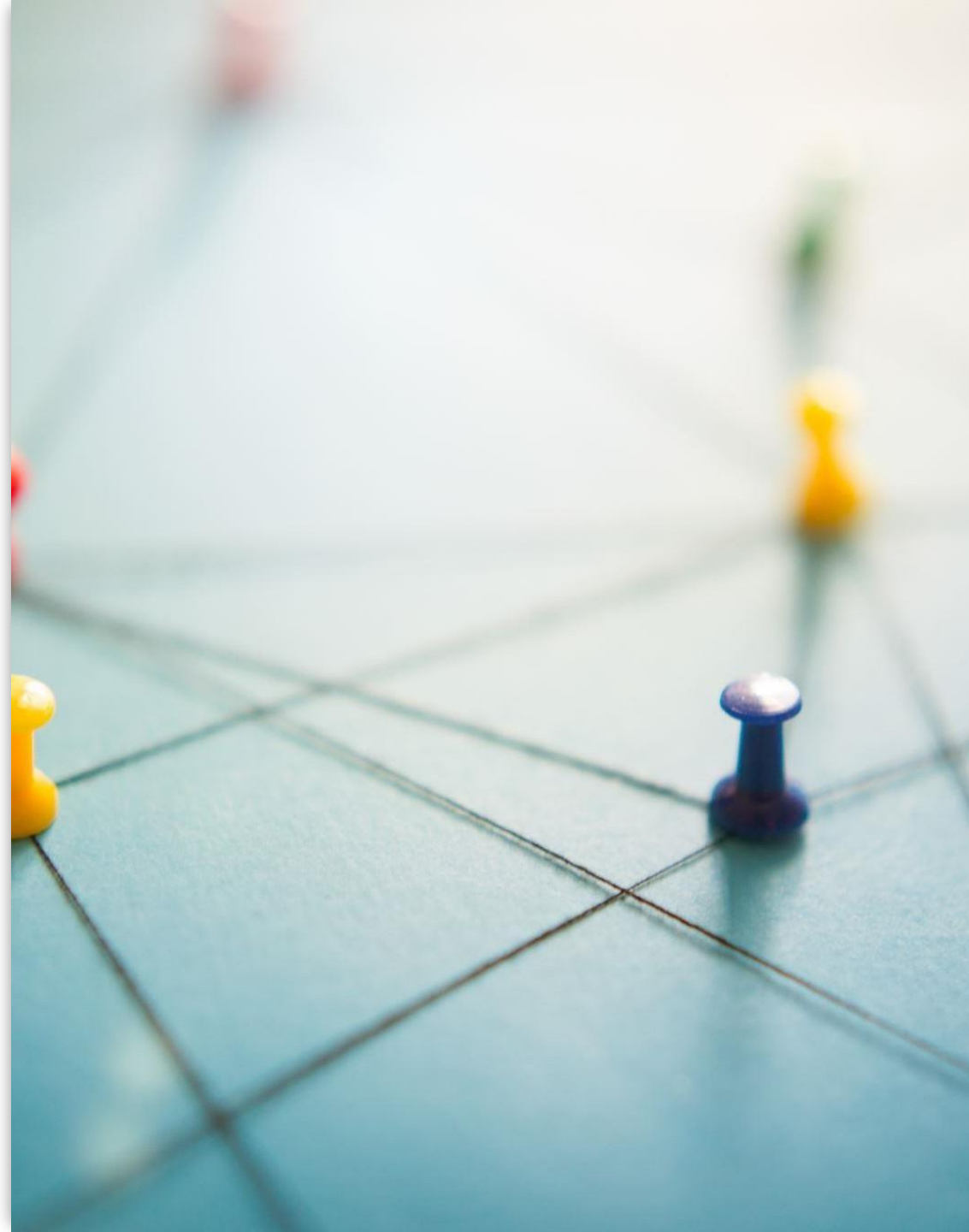
- Step 1
Identify all inputs, outputs, files, interfaces, and queries.
- Step 2
Classify them as **simple**, **average**, or **complex**.
- Step 3
Calculate **Unadjusted Function Points (UFP)**.
- Step 4
Compute **Technical Complexity Factor (TCF)**.
- Step 5
Final FP:
- $FP = UFP \times TCF$

Advantages s Limitations

- Advantages:
 - Language independent
 - Works early in project lifecycle
 - Based on user functionality
- Limitations:
 - Subjective estimation
 - Difficult to automate
 - Complexity of functions not always considered

Modern Techniques

- Modern estimation approaches include:
- Agile Story Points
Planning Poker
Relative Estimation
- These techniques focus on **iterative development and team collaboration.**



Tips for Better Estimation

- Break project into smaller tasks
 - Use historical project data
 - Consider team experience
 - Include buffer time
 - Update estimates regularly



Summary

- Software estimation helps plan **time, cost, and resources** for software projects.
- Key Techniques:
- COCOMO → Effort estimation
PERT → Time estimation
Function Points → Functional size estimation
- Accurate estimation helps ensure **successful project delivery**.